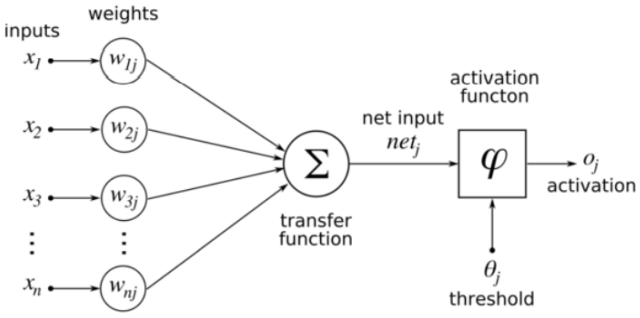
Lecture 9 - Artificial Neural Networks

An ANN is made up of layers of **perceptrons**. A perceptron is fed with binary information, and has a binary output as well (activation or not). Each input has an associated **weight**. A **threshold** determines the output of the perceptron. This threshold is usually set to 0 and a **bias node** is added that is equal to the negative of the threshold. The output of the perceptron is 1 if $w \cdot x + b > 0$, else 0.



Logic Gates

The following arrays are weights that model a perceptron that can perform as a logic gate. The first element is always the bias node.

AND - [-1, 1, 1]

OR - [-1, 2, 2]

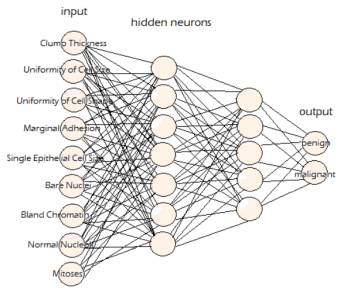
NOT - [1, -1]

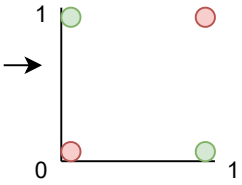
NOR - [1, -1, -1]

NAND - [2, -1, -1]

XOR - not possible (with a single perceptron).

Solution to the XOR problem:
Multi-Layer Perceptron





Activation Functions

Sigmoid Function: also called logistic activation function. Pro: small and gradual changes. Con: once neuron hits peak there's no learning anymore. Non-zero-centered means learning is harder (no negative values).

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Where $z = w \cdot x + b$.

Tanh: zero-centered version of the sigmoid function. Pro: easier optimization. Con: neuron peak problem still present. e^x requires comp. power.

$$\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Where $z = w \cdot x + b$.

Rectified Linear Units (ReLU): combination of two linear functions. Pro: computationally efficient (derivative below zero = 0, derivative above zero = 1), neuron does not hit peak. Con: non-zero-centered.

$$\sigma(z) = max(0, a)$$

Where $a = \sum_{i=1}^{n} w_i \cdot x_i + b$

Leaky ReLU: ReLU with small negative slope to fix the non-zero-centered problem.

$$\sigma(z) = max(0.01a, a)$$

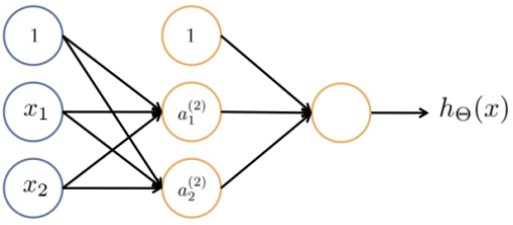
Why not linear functions? Non-linear functions allow deep learning models to approximate any continous function.

Softmax: use this function for a non-binary ANN.

$$softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_i}}$$

For $i = 0$ to k.

Forward Propagation



Let $a_i^{(j)}$ be the activation of unit i in layer j.
Let $\theta^{(j)}$ be the matrix of weights mapping from layer j to layer $j+1$.

$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2)$$
$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2)$$
$$h_\Theta(x) = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)})$$

Backpropagation

For each (x, y) in the training instance do:

1) Compute the forward propagation of x through the network and compute the activation a_u for every unit u

2) For each output unit k, calculate its error term δ_k:

$$\delta_k = a_k(1 - a_k)(y_k - a_k)$$

3) Calculate for each hidden unit h the error term δ_h:

$$\delta_h = a_h(1 - a_h) \sum_{k \in outputs} \theta_{h \to k} \delta_k$$

4) Update each network weight $\theta_{h \to k}$:

$$\theta_{h \to k} = \frac{1}{m} \cdot \Delta_{h \to k} + \lambda \theta_{h \to k}$$

Where $\Delta_{h \to k} = \Delta_{h \to k} + a_h \delta_k$.

Notes about ANN

- when initializing an ANN set the weights to random gaussian values in range [-0.05, 0.05].

- explanatory power of ANN is very limited. it also can't easily be transformed into another classifier e.g. decision tree.

- ANN is good when you need a high-dimensional input or output, and explanatory power isn't that important.

- ANN is useful for modelling complex and non-linear functions of numerical input and output.

- ANN is robust to noice, but there is a risk of overfitting (use validation sets to prevent).

- If the values in a hidden layer are exploding either below 0 or above 0, apply **batch normalization**; Re-centre the distribution so that the values are around 0. This is because layers will learn more independently and faster, and will also help with regularization (reducing overfitting).

- **Dropout:** random removal of neurons in each training round

- **Data Augmentation:** collect more data, transform instances, artificially generate new instances.